

Aplicación de propinas con porcentaje personalizado

Informe de resultados y conclusiones
Actualizado el 14 de septiembre de 2026

Se implementó la ampliación de Tip Time en Kotlin y Jetpack Compose. La aplicación permite ingresar un porcentaje de propina y elegir el redondeo hacia arriba. Las 13 pruebas de la lógica de cálculo pasaron; la compilación Android y la revisión en dispositivo quedan pendientes por la descarga incompleta de Gradle.

Objetivo y punto de partida

El objetivo de la actividad es ampliar la calculadora y documentar los cambios mediante control de versiones. El proyecto parte de la rama state del repositorio oficial de Google, revisión 19b7899. Esta base calcula la propina con un porcentaje fijo de 15 %. Se preservaron sus commits y su licencia Apache 2.0.

Cambios implementados

Siguiendo el codelab [1], se añadió un campo de porcentaje, un interruptor de redondeo, acciones Siguiente y Listo e iconos en los campos. La pantalla mantiene el desplazamiento vertical para acceder al contenido en horizontal. El campo reutilizable recibe su etiqueta, icono, valor y configuración del teclado como parámetros.

El estado de los dos campos y del interruptor reside en TipTimeLayout. Las funciones secundarias notifican cambios mediante callbacks. Se utiliza rememberSaveable para conservar estos valores al recrear la actividad. El contenido respeta los márgenes del sistema y el espacio ocupado por el teclado.

Cálculo y tratamiento de las entradas

La función tipValue obtiene la propina multiplicando el importe por el porcentaje dividido entre 100. Cuando el interruptor está activo, aplica ceil: 10.65 pasa a 11.00. Un resultado entero conserva su valor. calculateTip aplica el formato monetario de la configuración regional.

Como ampliación, parseNumber acepta punto o coma decimal sin separadores de miles. Los campos vacíos, negativos, no numéricos o no finitos se interpretan como cero. También se evita mostrar infinito si el cálculo desborda. Estas entradas no muestran un mensaje de error; incorporar validación visible es una mejora futura.

Resultados de la verificación

Se compiló TipCalculator.kt junto con TipCalculatorTest.kt usando el compilador Kotlin instalado con Android Studio. Se ejecutaron los 13 métodos de prueba con aserciones JUnit 4 mediante un lanzador Java por reflexión. La salida registrada fue "TOTAL: 13 tests passed". Este procedimiento valida el código de cálculo real, pero no la interfaz Compose.

Caso	Entrada o condición	Resultado observado
Porcentaje personalizado	100 al 20 %	20.00
Propina fraccionaria	71 al 15 % sin redondear	10.65
Redondeo hacia arriba	71 al 15 % con redondeo	11.00
Resultado entero	100 al 20 % con redondeo	20.00
Porcentaje cero	100 al 0 %	0.00
Entrada vacía	Texto vacío	0.0
Separador coma	125,50	125.5
Separador punto	125.50	125.5
Texto inválido	abc	0.0
Entrada negativa	-20	0.0
Valor no finito	NaN	0.0
Desbordamiento	Valores Double máximos	0.0
Formato de moneda	71 al 15 % con Locale.US	\$10.65

Los importes de esta tabla usan punto decimal para facilitar la comparación. El símbolo y la presentación monetaria en la aplicación dependen del dispositivo. La prueba de formato establece Locale.US temporalmente y restaura el valor original.

Alcance y comprobaciones pendientes

La ejecución estándar con Gradle, la generación de APK y lint no se completaron: falló la obtención de la distribución Gradle por conectividad/TLS. No se dispone de evidencia de ejecución en un emulador. Las pruebas de teclado, rotación, desplazamiento y accesibilidad deberán realizarse en Android Studio; no se presentan como aprobadas.

Historial y conclusiones

Trazabilidad de los cambios

Revisión	Contenido
4894ac6	Base oficial de la rama state publicada en GitHub
c2cb14a	Porcentaje personalizado, redondeo, teclado, estado y traducción
78db72e	13 pruebas de cálculo y flujo de verificación en GitHub Actions

El historial publicado reconstruye las etapas de la implementación mediante la conexión de GitHub y usa identificadores distintos de los commits locales. La documentación y las pruebas se incorporaron en 4edf89a. Los commits remotos se registran con la cuenta autenticada Axel421-yn; los commits locales originales conservan la autoría Codex. El bundle del ZIP mantiene el historial local original.

El código se publicó en la rama main de

[Axel421-yn/Aplicacion-de-Propinas-Axel-Adrian-Gavi-a-Gonzalez](#). No se agregó un README en ninguna de las etapas publicadas. El flujo de GitHub Actions está configurado para compilar, ejecutar pruebas y lint; sus resultados no se han confirmado en este informe.

Validación final en Android Studio

Abrir el proyecto con JDK 17 o compatible, instalar Android SDK 35 y sincronizar Gradle. Ejecutar assembleDebug, testDebugUnitTest y lintDebug. Después, introducir 71 como importe y 15 como porcentaje: la propina debe ser 10.65 y, al activar el interruptor, 11.00 en la moneda local. Comprobar que Siguiente mueve el foco, Listo cierra el teclado y el giro conserva los valores y permite desplazarse.

Conclusiones

La separación entre cálculo y presentación permitió verificar la lógica sin depender del emulador. Los resultados respaldan el porcentaje personalizado, el redondeo y el comportamiento definido para entradas vacías o inválidas. La elevación del estado permite que los componentes se reutilicen y que los cambios actualicen el resultado automáticamente.

La implementación cubre las funciones solicitadas en el código. Para confirmar su funcionamiento integral faltan la compilación Android y las comprobaciones de interfaz. El historial publicado en GitHub aporta la evidencia de trazabilidad requerida por la actividad. Como mejora posterior, conviene mostrar errores de entrada y añadir pruebas instrumentadas para teclado y rotación.

Referencias

[1] Android Developers. Calcula una propina personalizada. Consultado el 13 de septiembre de 2026. [Abrir el codelab oficial](#).

[2] Google Developer Training. Tip calculator, rama state. [Abrir el repositorio de partida](#).